The ML System Design Rubric

Scaler School of Technology | ML System Design and LLMOps | Week 5 — Framework Reference

This rubric is a **structured 10-step framework** for answering any ML system design question. Follow the steps in order. Each step builds on the previous one. Interviewers evaluate your answer on **structure, completeness, and ability to articulate trade-offs** — not on proposing the most complex model.

1 Clarify the Problem

Ask questions before designing. Define scope, constraints, and success criteria. **Key questions:** What must the system achieve? What metrics define success? Are there time, data, or resource constraints?

2 Context

Provide domain and business context. Why does this system exist? What is the business objective? Who are the users? What happens if the system fails?

3 Problem Statement

Formally define the ML problem. Frame it precisely: what is the input, what is the output, and what does the model need to predict or rank?

4 Define Requirements

Split into two categories. **Functional:** what the system must do (personalisation, real-time inference, feedback collection). **Non-functional:** how it must behave (latency SLA, scale, availability, privacy, maintainability).

5 Estimations

Back-of-envelope sizing. Estimate: number of users, request volume (QPS), data volume per day, storage requirements, compute needed. These numbers drive architecture decisions.

6 System Architecture

Design the end-to-end system. Decompose into: **(a) Data Collection** — sources, signals, ingestion patterns; **(b) Data Engineering** — ETL, storage, schema; **(c) Feature Engineering** — features, feature store; **(d) Preprocessing** — candidate generation, data prep; **(e) Model** — algorithm choice, training pipeline, registry; **(f) Post-processing** — re-ranking, filtering, business rules.

7 Evaluation: Offline and Online

Define how you measure success. **Offline:** precision@K, recall@K, NDCG, AUC-ROC on held-out data. **Online:** CTR, session length, business north-star metric. Define guardrails (what must *not* degrade).

8 A/B Testing

Design the experiment to validate the model in production. Define: treatment and control groups, sample size, duration, statistical significance threshold, and how you will detect bias (e.g. selection bias, novelty effect).

9 Deployment and Monitoring

Describe the production lifecycle. **Pipelines:** scheduling, orchestration (Airflow, Kubeflow). **Serving API:** REST/gRPC, inference server (Triton, Vertex AI). **Model controller:** version management, blue-green or canary rollout, rollback. **Dashboards:** latency, drift, business metrics.

10 Risk Management and Trade-offs

Identify what can go wrong and what you gave up. Address: data bias and fairness, model interpretability vs. accuracy, staleness vs. cost, cold-start, failure modes, and ethical implications.

How to use this rubric in an interview: Work through Steps 1–3 aloud with the interviewer — they may constrain the problem further. For Steps 4–6 spend the most time: this is where architectural thinking is assessed. Steps 7–10 differentiate senior candidates. You do not need to cover every sub-bullet — but you must be able to justify every decision you make.

Case Study: Real-time Music Recommendation System

Scaler School of Technology | ML System Design and LLMOps | Week 5 — Reference Solution

Step 1 | Clarify the Problem

Given problem: Design a real-time music recommendation system for a streaming service with **50 million active users**.

Constraints given: Personalised recommendations • p99 latency $<$ 100ms • 10,000 RPS peak • new songs visible within 24h • cold-start for new users.

Clarifying questions an interviewer would expect:

- How many songs in the catalogue? (*Assume: 100 million songs*)
- Is personalisation per-user or per-user-session? (*Per-user, updated nightly*)
- What availability SLA is required? (*99.9%*)
- Do we need to design the mobile app or just the ML system? (*Just the ML system*)

Step 2 & 3 | Context and Problem Statement

Context: A music streaming service competes for daily active listening time. Personalised recommendations are the primary driver of session length, discovery, and subscription retention.

Problem Statement: Build an ML system that, for each of 50M active users, returns a ranked list of 20 songs at the moment they open the app. The system must be personalised, operate under a 100ms p99 latency constraint, handle 10,000 requests per second at peak, expose new songs within 24 hours of upload, and gracefully serve new users with no prior history.

Step 4 | Define Requirements

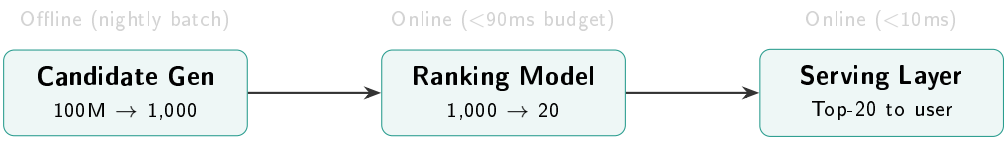
Requirement	Implication for design
Functional	
Personalised recommendations	Need per-user model or per-user candidate generation
New songs within 24h	Candidate generation must update at least daily
Cold-start for new users	Fallback strategy required before interaction history exists
Non-functional	
p99 latency $<$ 100ms	Cannot run full model inference over 100M songs in real-time
10,000 RPS peak	Serving must be stateless and horizontally scalable
99.9% availability	Serving layer must not depend on training pipeline being healthy

Step 5 | Estimations

- **Users:** 50M active users
- **Songs:** 100M in catalogue
- **Events:** ~500M play/skip events per day
- **Event data:** ~500 bytes/event $\approx$ **250 GB/day**
- **User feature vectors:** 50M $\times$ 128 dims $\times$ 4 bytes $\approx$ **25 GB in Redis**
- **Song embeddings:** 100M $\times$ 128 dims $\times$ 4 bytes $\approx$ **50 GB for ANN index**
- **Serving compute:** 10K RPS $\times$ 90ms $\approx$ 900 concurrent requests; 10–20 GPU serving pods
- **Training compute:** Nightly retraining on 30-day window $\approx$ 15B training pairs; 4–8 GPU hours

Step 6 | System Architecture

The key architectural insight: You cannot rank 100 million songs in real-time under 100ms. The solution is a **two-stage architecture**: offline candidate generation pre-filters to ∼1,000 candidates per user; online ranking ranks those 1,000 in real-time. This is how Spotify, Netflix, YouTube, TikTok, and Amazon all work.



6a Data Collection

What it does: Captures user events (plays, skips, likes, searches) and song metadata.

Signals to log: Plays, skip-at-second, like/save, search queries, playlist adds. Log *all* events including skips — what users skip tells you as much as what they play.

Key design: Kafka for real-time event streaming. Schema validation at ingestion (Great Expectations) prevents silent data corruption. Song metadata (genre, mood, BPM, artist) stored separately in a offline metadata store.

6b & 6c Data Engineering & Feature Engineering — Feature Store

What it does: Aggregates events into features for both training and serving.

Offline store (S3 + Parquet): nightly Spark jobs aggregate events into user-song interaction history. Point-in-time correct joins prevent data leakage — features for a training row at date D must only use data available *before* D.

Online store (Redis): sub-millisecond reads at serving time. Holds latest user feature vectors and pre-computed candidate lists. Refreshed every 4 hours.

Key features: User: recency-weighted play counts per genre/artist, skip rate, session length. Song: audio embeddings (spectrogram-based), popularity score, age since upload.

6d & 6e Preprocessing & Models — Training Pipeline

Candidate generation model: Two-tower neural network. User tower encodes interaction history; song tower encodes audio features + metadata. Both towers trained jointly to produce embeddings in a shared space. At inference, approximate nearest neighbour (ANN) search (FAISS) retrieves the 1,000 closest songs per user. Trained nightly on implicit feedback (plays), not explicit (likes) — plays are 100× more abundant.

Ranking model: LightGBM on user-song feature pairs. Features: user–song affinity score from two-tower, song popularity, recency, diversity penalty. Runs online over the 1,000 candidates.

Model Registry: MLflow tracks both models separately — changing the ranker does not require rebuilding the ANN index.

6f Post-processing — Serving Layer

Latency budget (p99 target = 100ms):

1. Network + load balancer: 5ms
2. Fetch candidate list from Redis: 5ms
3. Fetch user context features from Redis: 5ms
4. Run ranker on 1,000 candidates (Triton Inference Server): 75ms
5. Serialize + network: 10ms

Post-processing rules: Deduplicate (no same artist twice in top 5), inject novelty (1 unfamiliar artist per 5 recommendations), apply business rules (licensed content only, geo-restrictions). Serving pods are stateless and horizontally scalable.

Step 7 | Evaluation: Offline and Online

Type	Metric	Why it matters
Offline	Precision@20 / Recall@20	Fraction of top-20 recs the user actually engaged with in held-out period
	NDCG@20	Penalises relevant songs ranked lower; rewards correct ordering
	AUC-ROC	Binary click prediction quality for the ranking model
Online	**CTR (north star)**	Primary engagement signal; 7-day rolling average
	Session length	Total time spent after receiving recommendations
	Discovery rate	Fraction of recs that are new-to-user artists
	Diversity score	Mean intra-list cosine distance; guardrail against popularity collapse

Guardrail metrics: Diversity score must not drop >10% week-over-week. Cold-start CTR must be within 20% of warm-user CTR. A model that recommends the same 100 popular songs to everyone will have high CTR short-term but destroy long-term retention.

Step 8 | A/B Testing

Experiment design:

- **Unit of randomisation:** User ID (not session — avoids novelty effect contamination)
- **Treatment:** New model version. **Control:** Current production model.
- **Traffic split:** 10% treatment / 90% control initially; promote to 50/50 after 48h if no guardrail violations
- **Duration:** Minimum 14 days to capture weekly usage patterns and wash out novelty effect
- **Sample size:** With 50M users, 5M in treatment is statistically sufficient to detect a 1% CTR lift at α=0.05, power=0.80
- **Bias controls:** Holdout group (∼1% of users, never sees new model) for long-term measurement. Monitor for selection bias: treatment and control user distributions must match on age, device, and listening frequency.
- **Decision rule:** Promote if CTR improves ≥0.5% and diversity score does not drop >5%. Rollback if p99 latency exceeds 120ms or error rate rises >0.1%.

Step 9 | Deployment and Monitoring

Pipelines

- Airflow DAGs for nightly training and candidate refresh
- MLflow tracks every run: data version, code hash, hyperparameters, metrics
- Automated model evaluation gate: new model must beat production on NDCG@20 before promotion

Model rollout

- Blue-green deployment: new version serves 10% shadow traffic before full swap
- MLflow Registry stages: Staging → Production → Archived
- One-command rollback: demote current, promote previous

6 Key Metrics to Monitor:

1. **p99 serving latency** — alert if >120ms for 5 min
2. **Feature store freshness lag** — hours since last batch update
3. **Recommendation diversity** — mean intra-list cosine distance
4. **CTR** — 7-day rolling; alert on >5% drop week-over-week
5. **Cold-start vs. warm CTR gap** — should narrow over first 5 plays
6. **Candidate list staleness** — hours since last Redis refresh per user

Step 10 | Risk Management and Trade-offs

Decision	Choice	Alternative	Why this choice
Retrieval architecture	Two-stage (candidate gen + ranker)	Single ranker over all items	Cannot rank 100M songs in <100ms. Two-stage is the only feasible option at this scale.
Candidate freshness	Pre-computed, 4h refresh	Real-time ANN at serving time	4h refresh meets the 24h SLA and keeps latency predictable. Real-time ANN at 10K RPS is prohibitively expensive.
Ranker model	LightGBM	Small neural network	GBT trains faster, is more interpretable, and performs comparably on tabular ranking features.
Training feedback signal	Implicit (plays)	Explicit (likes)	Likes are sparse (<2% of plays). Plays are 100× more abundant — noisier but sufficient.
Cold-start strategy	Popularity + survey + fast online learning	No cold-start	A new user who sees nothing churns immediately. Popularity fallback is good enough for the first session.
Diversity	Intra-list penalty in ranker	None	CTR alone optimises for popularity collapse. Without diversity, long-term retention degrades.

Cold-Start Strategy (Tiered Fallback)

Cold-start is a **product problem**, not an ML problem. The fix is a tiered fallback strategy — not a better model.

User state	Strategy	Data source
0 interactions	Global popularity + editorial playlists	Aggregated play counts (all users)
1–4 interactions	Content-based: match audio features of played songs	Song metadata embeddings (genre, BPM, mood)
5–19 interactions	Collaborative + content hybrid	Partial interaction history + song embeddings
20+ interactions	Full personalised two-tower retrieval	Full interaction history in feature store

Failure Modes and Recovery

What breaks	How to detect	Recovery
Redis candidate cache fails	p99 latency spike; cache hit rate drops to 0	Fall back to global popularity; page on-call; rebuild cache from offline store
Nightly batch training fails	MLflow shows no new run; staleness alert fires	Serve previous model version; trigger manual retraining
Training-serving feature skew	Feature distribution diverges (Evidently); CTR drops without latency change	Pause new model promotion; compare offline/online distributions; fix pipeline
Ranker model degrades	CTR drops >5% week-over-week; diversity score falls	Roll back via registry; investigate training data quality
Cold-start loop	Cold-start CTR $\gg$ warm CTR; no personalisation transition	Check interaction logging; verify feature store updating for new users